

A Thorough Security Analysis of BLE Proximity Tracking Protocols

Abstract

Recent advances in Bluetooth Low Energy (BLE) and the ubiquity of mobile infrastructures promote the prevalence of BLE proximity tracking services (*e.g.*, Apple Find My and Samsung Find My Mobile) that use the proximity measured from other surrounding mobile devices (*e.g.*, smartphones). Accordingly, it raises severe security and privacy concerns that are inherent to the basis of the technique (*i.e.*, BLE) and the design of the proximity tracking protocol on top of it. Unfortunately, a systematic and comprehensive analysis of these protocols is still missing since the analysis of these protocols in existing research either focuses on a single participant in the service or lacks formal guarantees. As such, in this paper, we aim to fill in the missing piece by (1) recovering the closed-source protocol via reverse engineering; (2) building formal models based on reverse engineering; (3) extracting and formalizing the designed security goals of these protocols, and (4) formally verifying whether these security goals can be guaranteed. We reverse-engineered and verified two of the most popular real-world proximity tracking services, *i.e.*, Apple Find My and Samsung Find My Mobile. In total, our analysis reveals seven new vulnerabilities confirmed by related vendors, out of which, four CVE/SVE numbers are assigned, including three *high-severity* vulnerabilities. We also propose mitigations to the discovered vulnerabilities and formally confirm that all security goals can be achieved with our mitigations. At the time of paper writing, Samsung has fixed five vulnerabilities with our assistance.

1 Introduction

Due to the prevalence and ubiquity of mobile infrastructures and advancements in Bluetooth Low Energy (BLE), like energy-saving techniques, BLE proximity tracking has significantly expanded its mobile applications from indoor positioning to outdoor tracking in massive space. For example, Apple’s Find My [4] can track an object (*e.g.*, Apple AirTag [3]) globally using the proximity measured from other

surrounding mobile devices (*e.g.*, iPhones) via BLE. These BLE proximity-based services are becoming increasingly popular as millions of Apple’s AirTag and Samsung’s Galaxy SmartTag have been sold since launching in April 2021 [41], and the relevant market is expected to be billions of U.S. dollars [30]. As such, the security and privacy of these proximity tracking protocols are worth exploring since a security or privacy issue may affect millions of users.

Previous studies [22, 28, 56] have analyzed similar services and discovered privacy vulnerabilities in the services. However, these works are neither systematic nor comprehensive in that the analysis either focuses only on a single participant [22] or lacks formal guarantees [28, 56]. As such, a comprehensive and thorough study of these proximity tracking protocols is vital yet still missing.

To fill in the missing piece, in this paper, we conduct a comprehensive and systematic analysis of BLE proximity tracking protocols. To this end, we analyze two of the most popular proximity tracking protocols, namely Apple Find My [4] and Samsung Find My Mobile [43]. We first understand and recover the proximity tracking protocols by reverse engineering the firmware of the BLE trackers (*e.g.*, SmartTag) and the tracking apps on the smartphone, analyzing the traffic between the smartphone and the tracking server, and reading the specifications if available. Then, we read the specifications and/or other available documents to obtain and formalize the security goals that the proximity tracking protocol is designed to achieve. Lastly, we build formal models using SAPIC+ [15] for these protocols based on the results of reverse engineering and formally verify whether the designed security goals can be guaranteed.

In total, our analysis discovered seven new vulnerabilities (three during reverse engineering and three through formal verification) that allow an attacker to illegitimately rebind a tracker, which has been bound to a user’s account, to the attacker’s account or to illegitimately track the physical location of the tracker. We responsibly disclosed our findings to the corresponding vendors (*i.e.*, Apple and Samsung) who have confirmed our findings. Specifically, Samsung assigned

four CVE/SVE numbers to the vulnerabilities we disclosed, among which, three vulnerabilities are rated as *high* severity, and two vulnerabilities are rated as *moderate* severity. At the time of paper writing, Samsung has fixed five vulnerabilities with our assistance and we are informed that Apple will fix the vulnerability we reported in Fall 2024. Lastly, we also propose mitigations for the vulnerabilities we revealed and formally verify them to confirm that, with these mitigations, all the designed security goals can be achieved.

We summarize our key contributions as follows:

- **Understanding of the protocol.** To understand the closed-source proximity tracking protocol of Samsung, we reverse engineer the firmware of Samsung SmartTag and the corresponding tracking app on the smartphone and analyze the traffic between the smartphone and the tracking server. Through reverse engineering, we have recovered the full protocol of Samsung’s proximity tracking service and revealed three vulnerabilities.
- **Formal verification of the protocol.** To be comprehensive and further study the impacts of the vulnerabilities discovered during reverse engineering, we build formal models of Apple’s and Samsung’s proximity tracking protocols, extract the designed security goals of these protocols from their corresponding documents, and formally verify whether these security goals can be guaranteed. Our formal verification reveals three new vulnerabilities confirmed by Apple and Samsung, respectively.
- **Mitigations.** We propose mitigations to the discovered vulnerabilities, with the help of which, Samsung has fixed five of the reported vulnerabilities. We also perform a formal analysis of our proposed mitigations and confirm that the designed security goals can be achieved with our mitigations.

Our model and reverse engineering details are available publicly on GitHub [25].

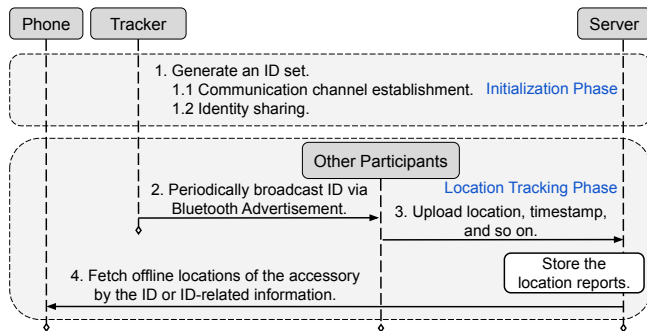


Figure 1: Overview of BLE proximity tracking services.

2 Bluetooth Proximity Tracking

In this section, we provide the basics of Bluetooth proximity tracking services and their general design security goals.

2.1 Service Overview

Bluetooth proximity tracking tracks an object (e.g., Apple AirTag, and Samsung SmartTag) by using the proximity measured from other surrounding mobile devices (e.g., iPhones or Samsung smartphone) via BLE. As such, it is also called crowd-sourced location tracking, and these services share a similar architecture, as shown in Figure 1. As the figure shows, the service involves a user’s smartphone that binds with a BLE tracker and inquiries the location of the tracker, a BLE tracker emitting BLE messages (e.g., AirTag), other participants that pick up the tracker’s messages and upload them to the server (e.g., smartphones of other users), and a cloud server that receives and stores the location of the tracker. We abstract the service into two phases, namely initialization and location tracking, as we detail below.

- **Initialization phase.** In this phase, the smartphone binds with the tracker by generating unique identities for the tracker. In general, this phase contains two steps, *i.e.*, channel establishment and ID sharing. Often, the communication channel established is protected using BLE’s just works pairing, or the security mechanism developed by the corresponding vendor, or both. In ID sharing, the smartphone shares the same ID key with the tracker, during which, the server may authenticate the tracker by some secret stored in the tracker while it was produced to protect the service from being abused.
- **Location tracking phase.** After initialization, the tracker will periodically broadcast BLE advertisements with a certain format so that other participants can distinguish the tracker from other BLE devices. In the advertisement, there is a field (ID field) generated using the tracker’s ID key, indicating that this advertisement is from the tracker corresponding to the ID key. The tracker usually adopts a rotation mechanism to maintain a certain degree of privacy by regenerating and changing the value of this field periodically. In the meanwhile, when another participant recognizes the format of the BLE advertisements broadcasted by the tracker, the participant picks up the advertisement and sends a location report, which includes the ID field of the advertisement, the physical location of this participant, timestamp, etc., to the server. As a result, the smartphone can send requests to the server to inquire about the location report of which the ID field corresponds to the tracker’s ID key. Lastly, the smartphone obtains the physical location of the participant who sends the location report. Since the participant is within the BLE range of the tracker, the smartphone can get the proximate physical location of the tracker.

2.2 Security Goals

We focus on three types of security and privacy goals of a proximity tracking protocol, namely confidentiality, authenticity, and anonymity. We will detail all security goals we examined in section 4.2.

- **Confidentiality.** An attacker cannot obtain certain data in plaintext, such as ID keys or secret locations.
- **Authenticity.** The smartphone and the tracker authenticate each other properly in the initialization phase and the server authenticates the smartphone properly when replying to requests for location reports.
- **Anonymity.** An attacker cannot increase or decrease the correlation of two or more items of interest, *e.g.*, BLE advertisements, after his observation [35].

2.3 Threat Model

In the initialization phase, we assume all the entities (*i.e.*, the tracker, smartphone, and server) are honest. In this case, we assume attackers have the capabilities defined in the Dolev–Yao model [20], meaning that attackers can intercept, eavesdrop, and inject messages transmitted in the Bluetooth physical channels between the tracker and the smartphone. Since the communication between the smartphone and the cloud server is well protected (*e.g.*, encrypted using SSL or TLS), we consider this communication channel as secure.

In the location tracking phase, besides assuming all entities are honest (the same as the initialization phase), we also consider the smartphone to be dishonest. For example, the adversary may steal or pick up a lost tracker and want to use it as her own tracker (*i.e.*, adding this tracker to her smartphone). In this case, we assume the adversary has the Dolev–Yao capabilities for the communication between the tracker and the smartphone and the communication between the smartphone and the server. As another example, (the smartphone of) the adversary may participate in the tracking system as other participants that pick up the tracker’s wireless signals and upload them to the server. In this case, we assume the adversary has the Dolev–Yao capabilities for communication between the tracker and her smartphone and between her smartphone and the server.

Lastly, we assume attackers are within the Bluetooth range when launching attacks on Bluetooth physical channels.

3 Understand Proximity Tracking Protocols

To perform security analysis for BLE proximity tracking protocols, we use two dominating vendors, *i.e.*, Apple and Samsung as examples to understand how the service works.

Apple Find My. We comprehensively understand the implementation protocols, via the previous work of Alexander

Heinrich et al. [28] on reverse engineering the Apple Find My protocol, the reverse engineering work of Adam Catley [19] for Apple AirTag, and the specification [2] that Apple released for Apple Find My Network Accessory in 2021. Based on the available documents and our experiments, we briefly describe the two phases of the Apple Find My service and provide detailed descriptions in Appendix C.

During its initialization phase, the user’s iPhone and her AirTag will collaboratively generate a master public key P , and two symmetric keys (SKN, SKS) for rotating P periodically. But for security reasons, only the iPhone can calculate the master private key d . Also, with the Apple public key stored in the AirTag, the Apple server and the user’s AirTag will collaboratively generate a shared secret *ServerSharedSecret* for secure communications between the Apple server and the user’s AirTag.

During the location tracking phase, other Apple devices (*e.g.*, iPhones) may discover the user’s AirTag by receiving its BLE advertisements, which contain its rotating public key P_i . Then, these Apple devices encrypt their locations using P_i and upload the location with the hash value ¹ of P_i with the SHA-256 hash function to the Apple server. After that, the user can fetch her location reports using the hash value as the index and decrypt the location using the corresponding rotating private key d_i .

Samsung Find My Mobile. Unlike Apple Find My, as we start our work, to the best of our knowledge, we are not aware of any reverse engineering effort or official documentation from Samsung regarding its Find My Mobile design or implementation. As such, to understand how Samsung Find My Mobile works, we reverse-engineered the related apps on a Galaxy S7 smartphone, a Galaxy S9 smartphone, and two SmartTags with firmware versions 01.01.26 and 01.02.06. In the following, our reverse-engineering efforts are primarily focused on Samsung Find My Mobile. We provide the detailed reverse engineering process and results in our repository [25].

3.1 Methods and Tools

We start our reverse engineering by capturing and analyzing the network traffic between the smartphone and the tracker and the traffic between the smartphone and the server to have a rough idea about what messages are exchanged. After that, we follow each exchanged message to dig deeper to understand how the message is generated and processed by reverse engineering the tracking apps on the smartphone and the firmware of the tracker. Lastly, we conduct dynamic testing to confirm our results of reverse engineering are correct and accurate.

Network analysis. To comprehend information exchanges, we monitor network interfaces in BLE proximity tracking protocols. We use the Wireshark [53] and a Bluetooth sniffer (*i.e.*, an nRF52832 Dongle [38]) to capture Bluetooth traffic.

¹This hash value is used as the index of the corresponding location report.

To capture the traffic exchanged between the smartphone and the server, we use the BurpSuite [36] to proxy and decrypt HTTPS traffic with self-signed TLS certificates. To this end, we disable certificate pinning on rooted smartphones.

Firmware extraction. Since the Nordic nRF52 series chips, which are used by SmartTags, are vulnerable to the voltage-fault injection attack [31], we successfully bypass the access port protection and enter the debug mode by launching this attack. In the debug mode, we can extract the firmware of a SmartTag. We extracted the related applications from Samsung smartphones (no root needed) via Android Debug Bridge (adb) using “adb pull” commands.

Code analysis. To understand the closed-source protocols, we perform both static and dynamic analyses. For static analysis, we use Binary Ninja [12] and JADX [24] to decompile the firmware and the tracking app on Android, respectively. In firmware analysis, we primarily focus on the verification and generation processes of the messages transmitted during the initialization phase and the construction process of the BLE advertisement during the location tracking phase. We first use FirmXray [50] to determine the base address of the extracted firmware. Then, based on the official documentation of Nordic Semiconductor [34], we locate related functions via specific system calls (SVC functions, which exist as enumerations in the API reference [34]), and perform further binary analysis. While reverse engineering for Android applications, according to the Android log captured via Android Studio [26], we first locate related codes via special strings and then track the call traces via function references, message handling, and intent calls. We also use dynamic analysis to facilitate reverse engineering. Specifically, when reverse engineering Samsung’s tracking apps on Android, we use Frida [21] to hook certain functions to get the input/output values of the function, which can greatly help us understand the underlying logic of the protocol.

Server API analysis. To comprehensively understand the interaction between the smartphone and the server, besides the traffic analysis, we also dynamically test the server APIs that handle HTTP GET and POST requests by sending crafted HTTP requests and responses². In detail, we first infer the functionality of each API based on our reverse engineering efforts of the firmware and tracking apps. We then dynamically validate our inference by sending various HTTP requests with carefully crafted fields to the Samsung server. For example, we can craft and send a set of requests by replacing account-related JSON fields in the HTTP request body with another account’s relevant data. Additionally, as a normal inference, certain server APIs have mandatory context relationships if they share some common fields in HTTP requests or HTTP responses. As such, we permute the HTTP request sequences

to test if we can bypass certain states and to understand the context relationship of each server API.

3.2 Demystify Samsung Find My Mobile

Initialization phase. As we discussed in Section 2.1, in the initialization phase, the Samsung smartphone first communicates with the SmartTag to generate the ID key. The smartphone establishes a BLE connection with the SmartTag using BLE’s Just Works pairing mode and communicates with SmartTag via the Generic Attribute Profile (GATT) read and write requests. After that, the initialization process performs the following four steps: (1) ECDH (Elliptical Curve Diffie-Hellman) PK (Public Key) exchange, (2) authentication, (3) authorization, and (4) ID generation.

In the remainder of this section, we detail the initialization process recovered by our reverse engineering effort, as Figure 2 shows. More details about the reverse engineering of the initialization phase can be found in our repository [25].

For ease of explanation, we refer to the phone as P , the tracker (i.e., the SmartTag) as T , and the server as S . During reverse engineering, we find Samsung writes a fixed private-public key pair into SmartTags during device manufacturing. Hence, at the beginning of the Samsung initialization phase, this key pair is shared between T and S .

In Step 1 of Figure 2, the tracker derives its private key via the SHA-512 hash function and some byte manipulation as described in our repository [25]. Also, the tracker shares its identifier and public key with the server, when being produced. As Figure 2 shows, in Step 1, P receives the hash value (*hashed_sn*) of T ’s MAC address from T . Subsequently, P generates a random number *rand* and sends *rand* and *hashed_sn* to the URL */identity/easysetup/blob* under S ’s domain, which is *client.smartthings.com*, using HTTP POST. As a response, P receives S ’s public key (*cloud_pub_key*) and forwards it to T . As we previously mentioned, S knows T ’s public key, and T also receives S ’s public key, T and S then perform ECDH to generate a shared secret (*DHKey*). After that, T and S can further derive a shared secret (*masterSecret*) following the procedure indicated by (6) in Step 1 of Figure 2. Note that all the exchanged data is in plaintext since P and T do not employ BLE’s encryption feature.

In Step 2, P and T first derive the same authentication key (*auth_key*) using equation $auth_key = kdf2(masterSecret, “bleAuthentication”)$, where $kdf2()$ is a hash function in SmartTag’s firmware. P and T then perform challenge-response-scheme mutual authentication using *auth_key*. Upon successful authentication, P and T generate a shared session key (*session_key*) to encrypt subsequent communications via “AES/CBC”.

In Step 3, P receives T ’s identifier, which is T ’s MAC address, and uses this identifier to query T ’s binding status by sending an HTTP GET request to the URL */chaser/tracker-s/lostmessage* of S ’s domain. If the response from S is *Tracker*

²We take ethical issues seriously. In this study, we took measures to minimize the impact on Samsung’s services by minimizing the test request packets, and we only tested our own devices with our own Samsung accounts.

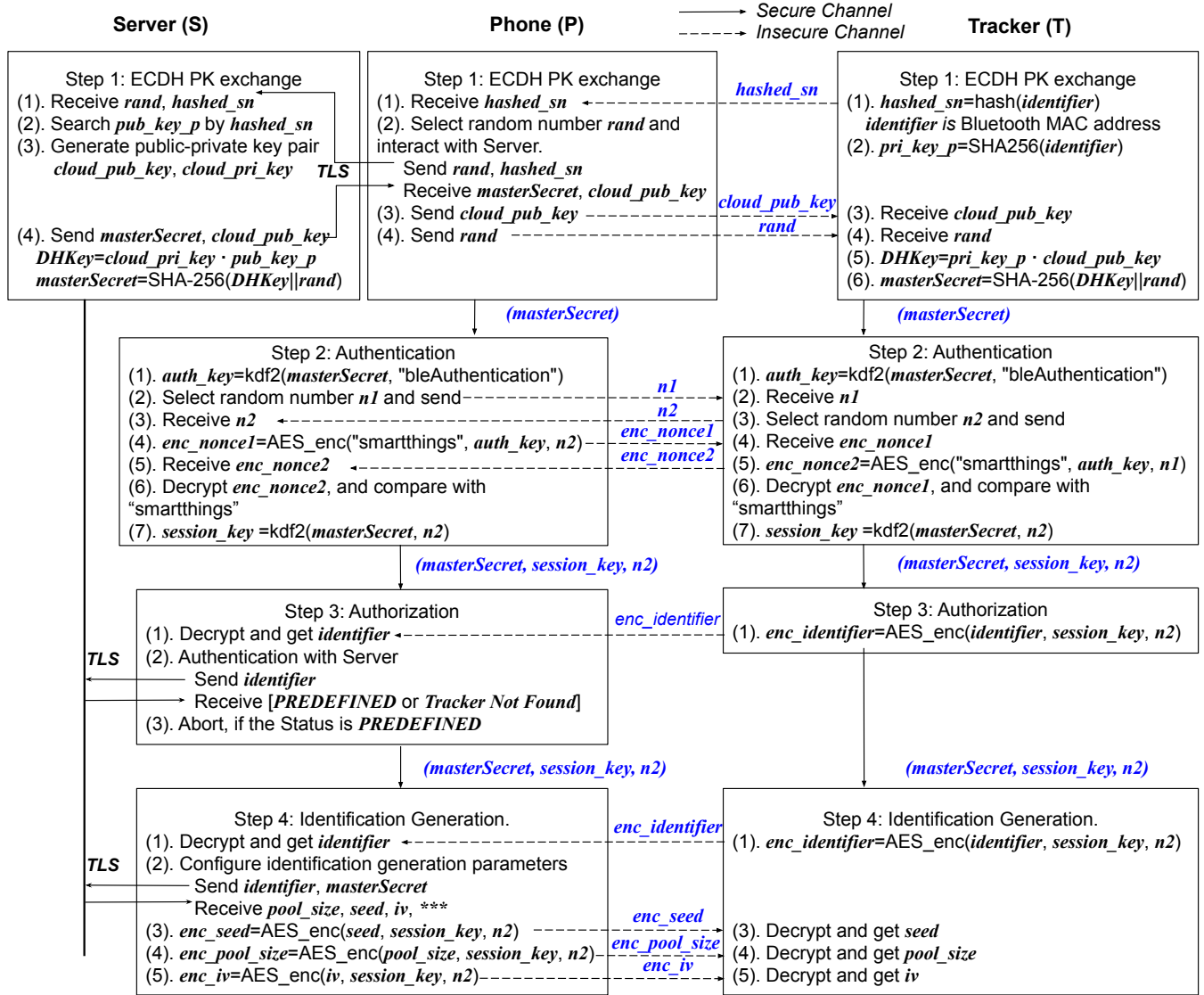


Figure 2: The initialization phase of Samsung Find My Mobile.

Not Found, indicating that *T* is not bonded yet, then *P* can continue to bind with *T*. Otherwise, if the response is *PREDEFINED*, meaning that *T* has been bonded with another Samsung account, then *P* will abort the binding.

In Step 4, *P* receives the ID generation parameters, including the privacy seed (*seed*), the pool size of SmartTag's ID (*pool_size*), and the privacy initial vector (*iv*) from *S* by sending an HTTP POST request to the URL */miniature/mobile* of *S*'s domain. Lastly, *P* relays these parameters to *T*, and the initialization phase finishes. After initialization, *P*, *T*, and *S* share the same ID generation parameters (*masterSecret*, *seed*, *pool_size*, *iv*).

Location tracking phase. SmartTags users can track their SmartTags with the help of other participants to discover the SmartTags and upload their location reports. Hence, we first

describe how other participants discover nearby SmartTags. Then, we present the details of how other participants upload location reports and how a SmartTag user fetches the location of her SmartTag. The overall process of the location tracking phase of Samsung Find My Mobile is illustrated in Figure 3. We provide more details based on our reverse-engineering efforts in our repository [25].

Discovering trackers. Samsung inserts a discovery service in its smart devices which constantly scans for nearby BLE advertisements using a filter (Service UUID: 0xFD69). When it finds a BLE advertisement with the Service UUID, it parses the payload of the BLE advertisement in a specific format as described in our repository [25].

Uploading location. As described in Step 1 of other participants in Figure 3, when receiving SmartTags advertisements,

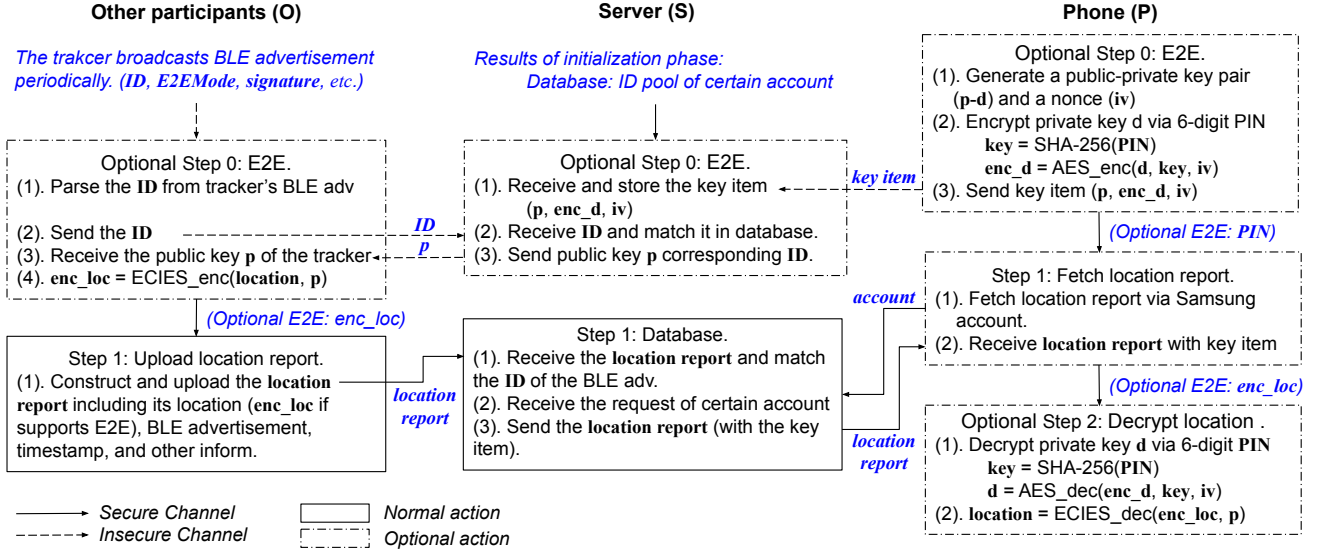


Figure 3: The location tracking phase of Samsung Find My Mobile.

a participant first parses the ID and E2E mode fields in the advertisement. If the E2E encryption is not enabled, the participant uploads a location report, which includes its location in plaintext, the BLE advertisement, and the timestamp to the server by sending an HTTPs POST request to URL `/geolocations` in a server domain. The domain of the server also depends on the region set in the advertisement.

Fetching location. After the initialization phase, the user can use her smartphone to log into her Samsung account and fetch the latest location of her SmartTag. As described in Step 1 of the phone in Figure 3, to fetch the location, the smartphone sends an HTTP POST request to URL `/installedapps/{installedapps_id}/execute` in the `api.smarththings.com` domain with the command URI `/trackers/geolocation` in the request body. If the E2E encryption is not enabled, the location report includes the tracker's location in plaintext.

(Optional) E2E encryption. Our analysis indicates that Samsung Find My Mobile supports end-to-end (E2E) encryption. As described in Step 0 of the phone in Figure 3, the smartphone can first generate a public-private key pair (p, d) and an initial vector (iv). Then, the smartphone derives a secret key (k) from a 6-digit PIN set by the user, and uses AES with k as the encryption key and iv as the initial vector to encrypt d . Lastly, the smartphone uploads p , encrypted d , and iv to the server by sending an HTTP POST request to `/installedapps/{installedapps_id}/execute`³ in the `api.smarththings.com` domain (which is different from S 's domain in Figure 2) with the command URI `/device/offline/key` in the HTTP request body.

³The `installedapps_id` is obtained from the Samsung server via a GET request to `/installedapps`.

As described in Step 0 of other participants in Figure 3, the participant first retrieves the above-mentioned public key from the server by sending an HTTP GET request to URL `/pubkeys?pid={ID}` in a server domain. The server's domain is different in different regions. For example, the request is sent to `chaser-na03-useast2.samsungiotcloud.com` if the region code is 10 (meaning the region is NA). Once the public key is received, the participant encrypts its location using the Elliptic Curve Integrated Encryption Scheme (ECIES) to encrypt its location.

As described in Step 2 of the phone in Figure 3, the smartphone decrypts the encrypted location report with the following steps: (1) derive the symmetric key k from the 6-digit PIN set by the user; (2) decrypt the encrypted private key d in the key item by k and iv ; (3) perform the same ECIES to decrypt the encrypted location report using the private key d .

3.3 Discovered Vulnerabilities

During our reverse engineering for Samsung Find My Mobile, we identified three vulnerabilities, as listed in Table 1, that violate some security goals described in Section 4.2.

V1: Authentication Bypass.

Vulnerability description. During our server API analysis, we find that the server did not authenticate the smartphone properly in the initialization phase, allowing the attacker to successfully bind a SmartTag, which was already bonded with another Samsung account, with her account.

To do this, the attacker first resets the SmartTag by following the instructions outlined in [42]. Then, the attacker performs Step 1 in Figure 2 to receive the `masterSecret` from the

Table 1: Summary of vulnerabilities discovered in reverse engineering.

No.	Vulnerabilities	Requirements	Severity
V1	Authentication Bypass	The attacker can access SmartTags and she has a rooted smartphone.	High
V2	Low ID Entropy	The attacker controls one or more BLE transceivers to cover a larger area.	Moderate
V3	Public Key Reuse	The attacker controls one or more BLE transceivers to cover a larger area.	Moderate
V4	Private Key Leakage	The stored encrypted private key on Samsung server is leaked.	Low
I1	Insecure Default Setting	The stored location on Samsung server is leaked.	Work as Intended
I2	Non-mandatory E2E	Samsung controls the server infrastructure as the service provider.	No Security Impact

server. After that, the attacker can skip Step 2 and Step 3 and perform Step 4 with the received *masterSecret*. Surprisingly, the new bond between the attacker’s Samsung account and the SmartTag overwrites the original bond between the victim user’s Samsung account and the SmartTag on the server. As a result, the victim user is not able to find the SmartTag in her account anymore.

V2: Low ID Entropy.

Vulnerability description. In Step 4 in the initialization phase, the smartphone receives the configuration of the ID Entropy (*pool_size*) from the server. We find that the *pool_size* is limited, making long-term device tracking available by collecting and matching the IDs.

In Step 4 (2) in Figure 2, after sending the HTTP POST request to */miniature/mobile* in the server’s domain, the server returns an HTTP response with a *privacyIdPoolSize* (which is the *pool_size* in Figure 2) field, of which, the value is 1,000. A device enabling the Samsung Find My Mobile service would broadcast a random ID and change it every 15 minutes and the *pool_size* is the entropy of the ID set. Due to the limited entropy, a device will frequently reuse its ID, making it trackable in practice. To track a victim device, an attacker can first collect most of the IDs used by this device by sniffing BLE advertising packets. The more IDs the attacker collects, the higher the chance that the attacker can identify the victim device. Through our analysis in Appendix A, if an attacker can collect 80% of IDs used by the victim device, which may take up to three weeks, the attacker can identify and track the victim device with a 96% successful rate within 30 minutes.

V3: Public Key Reuse.

Vulnerability description. If the E2E encryption is enabled, the smartphone will send a public key to the server, as we discussed in Section 3.2. Our analysis revealed that this public key is reused until the user explicitly resets the E2E encryption. As a result, an attacker can utilize this public key to tell whether the advertisements with different IDs are from the same SmartTag and thus further track the SmartTag.

By analyzing the *checkEncryptionData()* function in the *TrackerReportLocationTask* class of the *com.samsung.android.fmm* app, a participant will fetch a SmartTag’s public key from the server when enabling E2E encryption, as we briefly discussed in Section 3.2. In this case, an attacker can sniff the victim SmartTag’s advertising packets to obtain the IDs, and then fetch the public key corresponding to this ID from the server. If using different IDs obtains the same public key, it means that all these IDs are used by the same SmartTag. Accordingly, the attacker can identify and track the victim device.

V4: Private Key Leakage.

Vulnerability description. During reverse engineering, we find that when E2E is enabled, the user’s private key is first encrypted using a 6-digit PIN and then sent to the server, as shown in Step 1 of Phone (P) in Figure 3. Since this 6-digit PIN has a very low entropy, it can be easily recovered via a brute-force attack if the stored encrypted private key is leaked. Even Samsung itself can easily figure out the private key through a brute-force attack.

Therefore, when the stored location encrypted with the user’s public key on the Samsung server is leaked, the corresponding private key can be easily obtained through a brute-force attack, indicating that the protection of the user’s location is insufficient.

Minor Issue (I1): Insecure default setting. Though E2E encryption is supported by Samsung Find My Mobile, we find that it is disabled by default. As a result, the server can access all location reports in plaintext unless the user explicitly enables E2E encryption.

In Step 4 in Figure 2, the server responds to the smartphone with a configuration for the SmartTag. This configuration has a “tag.e2eEncryption” attribute (‘‘name’’: ‘‘tag.e2eEncryption’’) of which the default value is 0 (‘‘default’’: {‘‘value’’: 0}), indicating that E2E encryption is disabled. As such, if a user does not enable this feature explicitly, all location reports of her devices are uploaded to the server without extra encryption. In this case, the Samsung server can get the location report in plaintext and know the physical location of each device that does not enable the E2E feature.

Minor Issue (I2): Non-mandatory encryption in E2E mode. During our server API analysis, we found that Samsung Find My Mobile does not force the participants to upload the encrypted location reports in the E2E mode, allowing Samsung servers to know the tracker’s location even if it has

enabled the E2E function. In the E2E mode, the participants will not encrypt the location reports if they cannot get the public key of the tracker from the Samsung server in cases like network congestion. As a result, the participants will upload the location report of the tracker to the Samsung servers in plaintext. What is worse, there is no notification sent to the tracker user when the location report is sent in plaintext, leaving the user to believe that her privacy is still protected.

4 Formal Analysis of Proximity Tracking Protocols

We have understood the design of Apple’s and Samsung’s proximity tracking protocols by reading the specifications and reverse engineering. As a side product of reverse engineering, we have discovered three vulnerabilities affecting Samsung’s Find My Mobile service. Though reverse engineering can help reveal vulnerabilities, it does not guarantee completeness. Additionally, we want to explore further all possible impacts of the vulnerabilities discovered via reverse engineering. For example, besides the attack discussed in Section 3.3, what could be the security implications of reusing the public key? To achieve these two goals, we build formal models of Apple’s and Samsung’s proximity tracking protocols and conduct a formal analysis.

To do this, we first describe how we design and implement the model of proximity tracking protocols in SAPIC+. Then, we present the protocol’s security goals claimed by Samsung and Apple. Lastly, we detail how we model these claimed security goals and verify them using our models.

4.1 Modeling Protocol

As discussed in Section 3, the proximity tracking protocol has two phases, i.e., the initialization phase and the location tracking phase. We follow the steps described in Section 3 to build our model. Specifically, each entity (e.g., phone) is modeled as a *process*. Following our adversary model, we model the channels between each two communication entities (e.g., smartphone and tracker) as shown in Table 2. Then, the entire protocol is modeled as the parallel composition, denoted by “|”, of communicating entities. The detailed implementation of our model can be found in our code repository [25].

When modeling the server, there are two operations performed on the server: (1) querying whether a certain value has been stored in its database, and (2) inserting a certain value in its database. The database behaviors can be modeled using a global store in Tamarin or ProVerif (e.g., table). However, based on our experiment, when using tables to model the database, the verification cannot terminate in the settings of unlimited sessions.

To improve the verification performance, we skip the database operations and directly abstract the results of the

Table 2: Channel modeling of each two entities involved in BLE proximity tracking protocols. P: phone, T: tracker, O: other participants, S: server, MP: malicious phone, and MO: malicious participant.

Channels in initialization	Security	Channels in location tracking	Security
$P \longleftrightarrow T$	Insecure	-	-
-	-	$T \longleftrightarrow O$	Insecure
-	-	$O \longleftrightarrow S$	Secure
$S \longleftrightarrow P$	Secure	$S \longleftrightarrow P$	Secure
$S \longleftrightarrow MP$	Insecure	$MO \longleftrightarrow S$	Insecure

querying action into two types: Stored and Unstored, meaning that a certain value is stored and not stored in the database, respectively. To do this, we define two variables (*var_stored*, *var_unstored*). During the normal initialization phase, we insert *var_unstored* as a parameter into the tracker process, representing that the tracker has never been bound to an account. For the reset operation of the tracker, we model it by inserting *var_stored* as a parameter into the tracker process and restarting the initialization protocol, meaning that the tracker has been bound to an account.

Additionally, due to the limitation of symbolic verification tools, it is challenging to model the modulo operation perfectly. Therefore, we simplify the modeling of modulo according to the scenario where it is used. For example, Equation (1) is used in the Apple Find My location tracking phase.

$$\begin{aligned}
SKS_i &= KDF(SKs_{i-1}, 'update') \\
(u_i || v_i) &= KDF(SKs_i, 'diversify') \\
u_i &= u_i \mod (q-1) + 1 \\
v_i &= v_i \mod (q-1) + 1 \\
P_i &= u_i \cdot P + v_i \cdot G
\end{aligned} \tag{1}$$

We model it as follows⁴:

```

let SKSi = kdf(SKsi-1, update) in
let ui = u(kdf(SKsi, diversify)) in
let vi = v(kdf(SKsi, diversify)) in
let Pi = add(mul(ui, Pi-1), mul(vi, G)) in

```

4.2 Identifying and Modeling Security Goals

In the documentation of proximity tracking protocols [2, 5, 40, 44], Apple and Samsung claimed several security goals that their proximity tracking protocols achieve.

⁴*G* in Equation (1) indicates the generation point of the NIST P-224 elliptic curve. *Q* is the point on the P-224 curve, *exp* is the P-224 scalar. *mul(exp, Q)* means the scalar multiplication on the P-224 curve and *add(Q1, Q2)* means the point addition on the P-224 curve.

4.2.1 Samsung Find My Mobile

As such, in the following, we present and model the explicit properties we have identified from these documents.

P1: Confidentiality of location. Samsung stated that “All data in SmartThings Find is encrypted and protected, so the tag’s location is not revealed to anyone but you.” [40] Accordingly, the offline location should be confidential meaning that the attacker cannot get the offline location in plaintext. We then formalize *P1* as follows, meaning that whenever the *Secret(location)* action occurs at time point *i* (*@i*), the adversary does not know *location*. *K(location)* means that the adversary knows *location*. “All” and “exist” are universal, respectively existential quantifiers.

```
All location #i. Secret(location) @i
==> not Ex #j. K(location) @j
```

P2: Unlinkability. Samsung announced that “Randomized ID changes every 15 minutes” [40] and “Keep user information safe” [40]. Following [7], we model it as “Can the attacker distinguish between the scenario where all sessions use the same tracker and the scenario where each session uses a distinct tracker?”. Using the replication “!P” to capture an unlimited number of sessions (PIPl...), we model the “single tracker” scenario in the first process and the “distinct trackers” scenario in the second process. In this model, T represents the tracker, O represents the other participant, and S represents the server. The tracker’s identity is defined using “new variables” and is included in the parameters denoted by “*”.

equivLemma:

```
new variables; !(T(*) | O(*) | S(*))
!new variables; T(*) | O(*) | S(*)
```

P3: Authentication of tracker. “Only one SmartTag can be registered to an item at a time,” described in Samsung’s document [44]. We convert this property to “if a tracker was bonded with an account, it cannot be bonded to another account successfully”. We first model “a tracker bonded with an account” by inserting the identifier with the corresponding stored status “Stored” into the server process as parameters. To test whether the binding action is successful, we add a monitor process to compare the IDs individually calculated by the smartphone and tracker. Then, we activate the *paired(device)* event before we run the server process and activate the *pairSuccess(device)* event after the monitor process confirms that the IDs received from the smartphone and the tracker are the same.

Therefore, we verify P3 by asserting: if the *paired(device)* event occurs at time point *i*, the adversary can not activate the *pairSuccess(device)* event at time point *j* (*j > i*).

```
All device #i. bounded(device) @ i
==> not Ex #j. PairSuccess(device) @ j & i < j
```

We then describe and model the implicit properties that Samsung Find My Mobile should guarantee. The implicit properties refer to properties that are not explicitly required to

be held in the description of Samsung’s document (or Apple’s) but can lead to security or privacy implications otherwise.

P4: Confidentiality of ID. From the user point of view, a completed initialization phase ensures that the Samsung server and SmartTag computed the same shared ID, and it should also provide confidentiality of the shared ID. Similar to *P1*, we formalize the property (*P4*) as follows.

```
All ID #i. Secret(ID) @i ==> not Ex #j. K(ID) @j
```

4.2.2 Apple Find My

In the following, we first present and model the explicit properties we have identified.

P5: Confidentiality of master private key. “Because the private key is stored only on the owner device, the location information is accessible only to the device owner.” [2], Apple says. Similar to *P1*, we then formalize the properties of the master private key *d* (*P5*) as follows.

```
All d #i. Secret(d) @i ==> not Ex #j. K(d) @j
```

P6: Confidentiality of master public key. “P itself must never be sent out and must be stored in a secure location” [2], Apple says. In contradiction with what its name “public key” suggests, the master public key should be kept as a secret according to Apple. Similar to *P1*, we then formalize the properties of the master public key *P* (*P6*) as follows.

```
All P #i. Secret(P) @i ==> not Ex #j. K(P) @j
```

P7: Confidentiality of symmetric key. “Both the owner device and the accessory compute the final symmetric keys *SKN* and *SKS* as the 64-byte output of ANSI-X9.63-KDF(*x(P)*, *r || r'*), where *SKN* is the first 32 bytes and *SKS* is the last 32 bytes.” [2], Apple says. Accordingly, the final symmetric keys (*<SKN, SKS>* as *P7*) should be confidential meaning that the attacker cannot get the offline location in plaintext. Similar to *P1*, we formalize the properties (*P7*) as follows.

```
All symkey #i. Secret(symkey) @i
==> not Ex #j. K(symkey) @j
```

P8: Confidentiality of shared secret. “Additionally, the accessory and the server generate a shared secret [2]. The shared secret is used to derive a key and protects requests related to obtaining lost mode information: secret shared with server *ServerSharedSecret*, symmetric key for pairing session *K1*, symmetric key for queries with serial number *KSN*.” [2], Apple says. Similar to *P1*, we formalize *P8* as follows.

```
All shared_secret #i. Secret(shared_secret) @i
==> not Ex #j. K(shared_secret) @j
```

P9: Confidentiality of location. “Apple server receives encrypted location data from Finder devices and temporarily stores it. Only the owner devices can decrypt and read raw locations from the encrypted data. Apple cannot read this information” [2], Apple says. Similar to *P1*, we then formalize the property (*P9*) as follows.

```
All location #i. Secret(location) @i
```

```
==> not Ex #j. K(location) @j
```

P10: Authentication of tracker. Apple announced that “An accessory will not be able to Find My network pair if it is paired to an owner device with a different Apple ID.” [5]. Similar to P3, we formalize the property (P10) as follows.

```
All device #i #j. bounded(device) @ i
==> not Ex #j. PairSuccess(device) @ j & i < j
```

P11: Unlinkability. Apple stated that “The accessory must derive primary and secondary keys from the public key P generated at pairing time.” [2]. However, when modeling the unlinkability of the Apple Find My protocol, a chained rotation mechanism of a public key should be modeled. With this mechanism, the public key of the current session is calculated based on the public key of the previous session. Similarly, the public key of the next session is computed using the current session’s public key. It is challenging to model the chained rotation mechanism with an unbounded number of sessions due to the dependencies between each session. So, following the usage of Sapic+ that the unbounded sessions will be translated into three sessions, we model the unlinkability of the Apple Find My with three consecutive sessions as follows:

```
equivLemma:
  new variables; (T(v0) | O(*) | S(*) | T(v1) | O(*)
| S(*) | T(v2) | O(*) | S(*)
!new variables; T(*) | O(*) | S(*)
```

5 Evaluation

Once we have the claimed properties formalized, we can use our model to verify whether these properties can be guaranteed. We run our verification by combining the advantages of state-of-the-art symbolic verification tools, i.e., ProVerif [37], Tamarin [48], and DeepSec [17]. In general, ProVerif offers better automation and speed than Tamarin, while Tamarin can provide a more precise model of Diffie-Hellman key exchange. DeepSec overperforms ProVerif and Tamarin in verifications of privacy properties, e.g., indistinguishability.

To take advantage of the strengths of each tool, we verify anonymity (unlinkability) using DeepSec and verify the reachability properties (confidentiality and authentication) using TAMARIN and ProVerif. We first verify these reachability properties using ProVerif with a timeout of 30 minutes, because there is no need to check it with the more precise DH theory of Tamarin if ProVerif finds an attack. If ProVerif fails to identify any violations or times out, we continue our verification using Tamarin with a timeout of 24 hours to strengthen the trust. For anonymity, we find that Sapic+ currently only supports automatic translation of equivLemma into session equivalence of Deepsec [17], which is stricter than trace equivalence and faster to prove. As such, when verifying anonymity, we first verify session equivalence in Deepsec based on Sapic+’s automatic translation. If Deepsec concludes “not session equivalent”, we then manually modify

the translated code to verify trace equivalence using Deepsec. We will further discuss this in Section 7. We also run the verification iteratively to comprehensively analyze the protocol. After finding a property violation, we fix it and perform a new round of analysis for this property on the fixed model. We repeat this process until all properties hold.

To improve the verification performance, especially in the settings of unlimited sessions, where the verification may not terminate, we introduce extra lemmas, options, or simplifications of our model to speed up the verification process. In detail, we introduce “not attacker(X)” to help ProVerif simplify the clause set by removing clauses that contain X in hypotheses, resulting in performance improvements. For Tamarin, we introduce some source lemmas in our models to mitigate partial deconstructions that complicate proofs, and use the command line option “-heuristic” to help Tamarin terminate the case distinction precomputation. Due to the complication of the Apple Find My initialization phase, we simplify the modeling of BLE Just Works pairing by treating the communication channel constructed by the BLE Just Works pairing as an insecure channel. We believe it is a reasonable simplification as the Bluetooth Specification specifies that the Just Works pairing method is vulnerable to MitM attacks and thus not secure [13].

In total, we have verified 17 properties and identified eight violations. We show the results and the runtime of property verification (in seconds) in Samsung Find My Mobile and Apple Find My in Table 3. All our experiments were performed on a 160-core Intel(R) Xeon(R) Platinum 8383C CPU @ 2.70GHz server with 2TB of RAM. Overall, the initial verification required less than 50 minutes with a 160-core parallelization.

Table 3: Results of the security properties verified in Samsung Find My Mobile and Apple Find My. P3+ indicates the P3 property in the model with V1 fixed. T : property holds. F : property can be violated.

Property	ProVerif	Tamarin	DeepSec	Vulnerability
P1	F(1s)	-	-	I1, I3
P2	-	-	F(1s)	V3
P3	F(1s)	-	-	V1
P3+	T(1s)	F(25m46s)	-	V5 (new)
P4	F(5m45s)	-	-	V6 (new)
P5	T(1m20s)	T(1m11s)	-	-
P6	F(1m20s)	-	-	V7 (new)
P7	F(1m20s)	-	-	V7 (new)
P8	T(1m20s)	T(1m11s)	-	-
P9	-	F(3s)	-	I4
P10	T(1s)	T(29s)	-	-
P11	-	-	T(1s)	-

5.1 Discovered New Vulnerabilities

During our verification, we reveal three new vulnerabilities, affecting Apple Find My or Samsung Find My Mobile protocols, as listed in Table 3. In the remainder of this section, we detail each of the newly discovered vulnerabilities and how to exploit them to launch attacks.

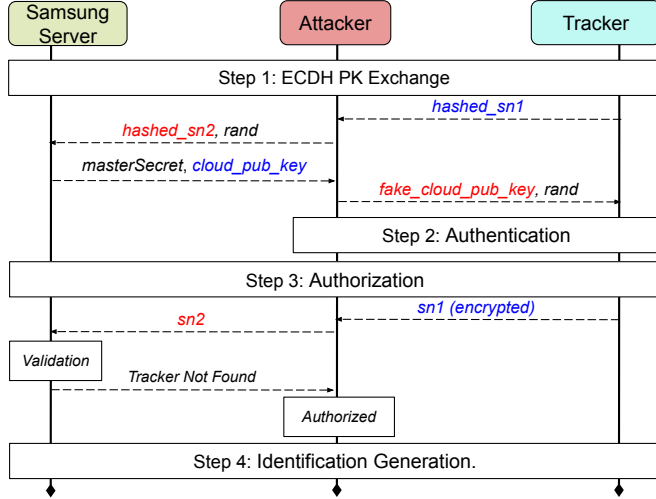


Figure 4: Procedures to launch the authentication bypass attack against Samsung Find My Mobile.

V5: Authentication bypass.

Vulnerability description. Our formal analysis reveals that $P3$ can be violated, and with further investigation, this violation corresponds to the vulnerability V1 we discovered during reverse engineering. After fixing V1 (we will further discuss in Section 6), our analysis indicates that $P3$ can still be violated. With the investigation of the attack trace, we find this violation is corresponding to a new vulnerability. Similar to V1, this vulnerability also allows an attacker to bypass the authentication procedure in the Samsung server in the initialization phase. As a result, the attacker can successfully bind a SmartTag, which has been bonded with another Samsung account, with her account.

To achieve this, the attacker first resets the SmartTag following the instructions [42] and starts the pairing procedure using her smartphone. After that, the attacker can follow the steps illustrated in Figure 4 to launch the attack. Specifically, the attacker intercepts the hash value ($hashed_sn1$) received from the SmartTag and sends a fake hash value ($hashed_sn2$) to the server. Since the server checks the binding status of a SmartTag based on the hash value, using a fake hash value can trick the server into believing that this SmartTag is not bonded to any Samsung account. Then, the attacker receives the server’s public key ($cloud_pub_key$) from the server, computes a faked public key (pub_f) with Equation (2) based

on the SmartTag’s private key (pri_key_p)⁵ and a fake private key (pri_key_f), and sends it to the SmartTag. With the fake public key, both the Samsung server and the SmartTag will derive the same *masterSecret*, as we will prove in Appendix B. Once the same *masterSecret* is derived, the attacker can continue the initialization process and successfully bind this SmartTag with her account. The detailed attack steps can also be found in our repository [25].

$$pub_f = pri_key_p^{-1} \cdot pri_key_f \cdot cloud_pub_key \quad (2)$$

V6: ID Leakage in Samsung Find My Mobile.

Vulnerability description. Our formal analysis also reveals that $P4$ can also be violated in the initialization phase of SmartTag. Accordingly, an attacker can obtain the user ID and further track the physical location of a SmartTag.

To launch the attack, the attacker needs to sniff the entire communications of the initialization phase. Since SmartTag does not use BLE pairing, like Just Works, the data exchanged in Step 1 and Step 2 in Figure 2 is in plaintext. The attacker can obtain the server’s public key ($cloud_pub_key$), the SmartTag’s MAC address⁶ and *rand*. Then, the attacker can derive the SmartTag’s private key (pri_key_p) from its MAC address and compute the Diffie-Hellman shared secret key (*DHKey*) with $cloud_pub_key$ and pri_key_p . With *DHKey* and *rand*, the attacker can derive the *masterSecret* as $SHA256(DHKey || rand)$. Further in Step 2, having the plaintext of $n2$, the attacker can derive the session key (*session_key*) as $kdf2(masterSecret, n2)$, where $kdf2()$ is a hash function implemented in SmartTag’s firmware. Later in Step 4, the attacker can decrypt all the encrypted communication using *session_key* to get *seed*, *pool_size*, and *iv*. As a result, the attacker can calculate the ID set of the SmartTag using all ID generation parameters (i.e., *masterSecret*, *seed*, *pool_size*, and *iv*). With this ID set, the attacker can identify this SmartTag and perform long-term device tracking. The detailed attack steps can be found in our repository [25].

V7: ID Leakage in Apple Find My.

Vulnerability description. Our formal analysis also reveals that $P6$ and $P7$ can also be violated in the initialization phase of the Apple tracker (e.g., AirTag). Accordingly, an attacker can obtain the master public key P and rotation symmetric keys $\{SKN, SKS\}$ and further track the physical location of the tracker by calculating its rotation sequence of the temporary public key (as its ID).

To ease understanding, we briefly present the collaborative ID generation process in Apple Find My. For ease of explanation, we refer to the iPhone as P , the tracker (i.e., the AirTag) as

⁵A SmartTag’s private key is derived from its MAC address. $pri_key_p = kdf1(MAC)$, where $kdf1()$ is a hash function in SmartTag’s firmware.

⁶Before initialization, a SmartTag uses its permanent MAC address, which is also the identifier in Figure 2, to broadcast advertising messages.

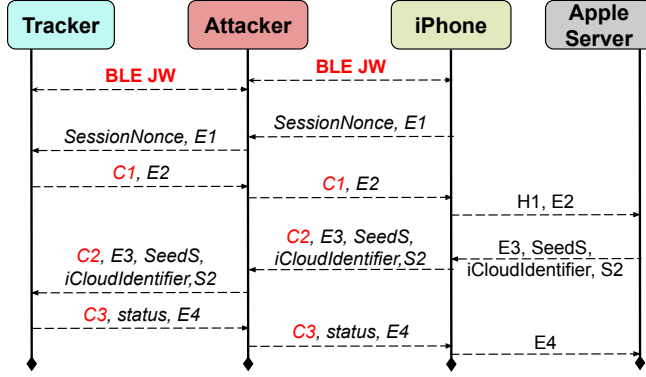


Figure 5: Procedures to launch the ID leakage attack against Apple Find My.

T, and the server as S. P generates its private-public key pair (s', S') and a random value (r') . T generates its private-public key (s, S) and a random value (r) . T first calculates the hash value $C1 = \text{SHA256}(s, r)$ and sends $C1$ to P . Then, P sends $C2 = (S', r')$ to T . Subsequently, T sends $C3 = (s, r)$ to P . After P verifies $C3$ is valid, P and T finish the ID generation process by calculating the master key item as described in Equations (3) and (4). Only P can calculate the master private key d as described in Equation(5).

$$P = S' + s \cdot G \quad (3)$$

$$(SKN || SKS) = \text{KDF}(x(P), r || r') \quad (4)$$

$$d = s + s' \quad (5)$$

To achieve this, as illustrated in Figure 5, the attacker first acts as the MitM to successfully pair with AirTag and the iPhone using BLE Just Works pairing. Then, the attacker records and replays subsequent messages. Through record-and-replay, the attacker obtains all the parameters, *i.e.*, the iPhone’s public key (S') and the random value (r') in $C2$, the AirTag’s private key (s) and the random value (r) in $C3$, needed to calculate the rotation sequence of the temporary public key.

Then the attacker can compute the rotation sequence of the tracker’s temporary public key as described in Appendix C.

Minor Issue I3 & I4: Flawed E2E. Although Samsung’s Find My Mobile with E2E mode and Apple’s Find My provide privacy protection, by exploiting the corresponding two violations (I3 and I4, respectively), Apple and Samsung can still access users’ location information in certain situations. In Samsung’s Find My Mobile, an attacker can exploit I3 by modifying the E2E mode in the tracker’s advertisements to NoE2E mode. As a result, innocent other participants will still pick up such advertisements and upload their location reports in plaintext to the server. Accordingly, the Samsung server is able to know the physical location of the tracker even though it enables the E2E mode. In Apple Find My, the tracker may use the generator of the Elliptic-Curve Cryptography (G) as

its public key, of which the corresponding private key is 1. In this case, since the private key is easy to guess, the Apple server can decrypt the location reports using this private key.

5.2 Rediscovered Vulnerabilities

During our formal analysis of Samsung Find My Mobile protocol, we rediscovered two vulnerabilities and one minor issue (*i.e.*, V1, V3, and I) that are revealed by reverse engineering in Section 3. Since we build our model using symbolic verification tools, which do not support brute force attacks, our model cannot rediscover V2, V4, and I2.

5.3 Responsible Disclosure

We responsibly disclosed our findings to the corresponding vendors. In detail, Samsung has confirmed our findings, and assigned five SVE numbers to the vulnerabilities we revealed (*i.e.*, SVE-2022-1178 for V1, SVE-2022-1176 for V2, SVE-2022-1177 for V3, SVE-2022-1179 for V5 and V6, and SVE-2022-1180 for V4), and concluded that the severity of V1, V5, and V6 is *High*, the severity of V2 and V3 is *Moderate*, and the severity of V4 is *Low*. However, for I, Samsung said “In order to satisfy both security and usability, the concept is determined by various factors. For this reason, we turned off the ‘encrypt-offline-location’ by default and provided an option for advanced feature”. Also, for V4, Samsung decided to keep current design concept, as the modification greatly degrades usability. For I3, Samsung said, “This report will be forwarded to the respective development team to be patched in future software maintenance releases.” Samsung has patched V1, V2, V3, V5, and V6. Additionally, Samsung fixed V5 and V6 by revising their factory process to inject random values for the private key, but could not apply the patch with their products that have already been manufactured. We also reported V7 to Apple in November 2022. We are informed that Apple is planning to fix the vulnerabilities we discovered in an update in Fall 2024.

6 Mitigation

In this section, we propose mitigations for the vulnerabilities we discovered and formally verify our mitigations.

Mitigate V1. We propose to check the bound status of the SmartTag being paired by its identifier, in step (4) of the initialization phase of Samsung Find My Mobile as shown in Figure 3.2. If the checking result is that the SmartTag is bound to another Samsung account, the server aborts the pairing procedure and responds with *PREDEFINED*.

Mitigate V2. Since the *pool_size* (the size of the randomized ID set) is a 2-byte long unsigned integer, we propose increasing it from 1,000 to 65,535 (the largest number with a 2-byte length) to increase the time required for collecting

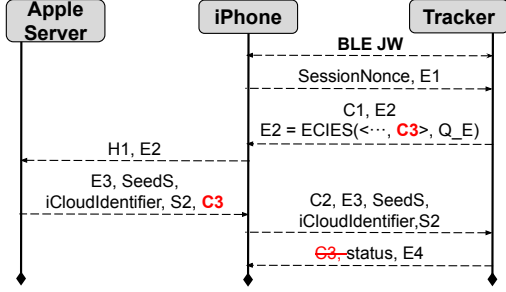


Figure 6: The mitigation of Apple Find My’s initialization protocol.

the randomized IDs. While, in theory, it is still vulnerable to passive eavesdropping attacks, in reality, it is impractical. As we proved in Appendix A, with a *pool_size* of 65,535, the average time for an attacker to collect 80% of IDs is more than 3 years, making the device tracking extremely impractical.

Mitigate V3. We propose to perform an anonymous rotation mechanism for the public key as follows, with A representing the Samsung server and B representing the Samsung user. Firstly, B generates a master private-public key pair (d_0, P_0) and a rotation parameter rp , encrypts the private key d_0 using the PIN code and a nonce iv , and uploads the key item (enc_d_0, P_0, iv, rp) to A. Then, A and B rotate P_0 by rp . We call the derived function outputs u_i and v_i . Finally, A and B can compute $P_i = u_i * P_{i-1} + v_i * G$. Only B can compute the $d_i = u_i * d_{i-1} + v_i$.

Mitigate V4. We propose to remove the restrictions on the number of digits and constituent characters of the password.

Mitigate I1. We propose to force Samsung users to enable the E2E function. Thus, when Samsung users help to track SmartTags and fetch the latest location of their SmartTags, Samsung servers cannot obtain the location in plaintext.

Mitigate I2. For I2, we propose to change the location uploading process in the participant’s device: when the fetching process of the public key gets wrong, suspend the reporting process for a while. Until the public key of the tracker is correctly obtained, construct and upload the location report.

Mitigate V5 and V6. We recognized that both vulnerabilities are caused by the inverse of the SmartTags’ private key, and proposed to inject the random public-private key pair when manufacturing SmartTags. Samsung has adopted our mitigation to revise their factory process, but Samsung cannot apply the patch to the products that have already been produced.

Mitigate V7. We propose changing the interactions between the phone and the tracker as shown in Figure 6, which is different from the original Apple initialization protocol as described in Appendix C. In detail, we insert C3 into the generation of E2, which is encrypted by Apple’s public key. Then, the server decrypts C3 and sends it to the phone.

Mitigate I3. The mitigation of I1 can also prevent I3. If the E2E mode is forced to be used, the location report in plaintext will not be uploaded.

Mitigate I4. To mitigate I4, we propose that the server collect the public key generated during the initialization phase and perform an additional check on the participant’s side to ensure the detected public key is valid.

We have performed a formal verification for the protocols with all of our mitigations and proved that all properties (*PI - P11*) hold. Overall, the verification of our changes required less than 5 minutes with a 160-core parallelization. We present the fixed models and results in our repository [25].

7 Discussion

Comparison with existing SmartTag reverse engineering work. Nevertheless, compared with the existing reverse engineering work of Samsung Find My Mobile [56, 57], our focus is on how Samsung Find My Mobile works, while [56, 57] focused on the smartTag itself. Among all the issues we identified, V2 and I1 are similar to the issues revealed in [56, 57]. We discovered other issues (V1, V3, V4, V5, V6, I2, I3, I4), which [56, 57] did not.

Reverse Engineering and Formal Analysis. In this work, we use reverse engineering to recover the closed-source Samsung Find My Mobile protocol and formally verify it to perform a comprehensive and systematic analysis of the Samsung Find My Mobile protocol. During reverse engineering, some vulnerabilities or security risks can be revealed, as we did in Section 3. However, reverse engineering cannot provide completeness. Additionally, for a vulnerability identified by reverse engineering, we are not able to comprehensively evaluate its impact (e.g., find all possible ways to exploit this vulnerability). As such, after recovering the protocol via reverse engineering, we use formal analysis, which is known for comprehensiveness, to further analyze the protocol in a comprehensive and systematic manner.

Analysis Granularity. We do not model finer operations within the cryptographic primitives (e.g., AES_CBC and SHA-256) and assume these cryptographic primitives are secure. Our assumptions align with the assumption of symbolic modeling and other existing research [8].

Tamarin, ProVerif and SAPIC+. Tamarin [48] and ProVerif [37], two state-of-the-art generic symbolic verification tools, have their advantages and disadvantages. In general, ProVerif is faster than Tamarin, while Tamarin can provide a more precise model of DH key exchange [15]. As such, Tamarin can discover some vulnerabilities that cannot be revealed by ProVerif, such as V5. To utilize the advantages of both tools, we design and implement our model in SAPIC+, which can convert the model implementation to both Tamarin and ProVerif soundly. Combined with the iterative verification, as discussed in Section 5, we can verify the protocol

efficiently and soundly. However, since we build our model using symbolic verification tools, which do not support brute force, our model cannot rediscover V2.

7.1 Limitations

Extensibility and other BLE proximity tracking protocols. We focus on the BLE proximity tracking protocols provided by the smartphone manufacturer because they can provide such services without the need for third-party support. Besides, Apple and Samsung, there are other vendors (e.g., Huawei, Tile, and Google) that provide proximity tracking services. We encountered several difficulties in our preliminary experiments, such as being unable to extract the firmware from the tracker and to extract the proximity tracking app from the smartphone. As such, analysis of these protocols is not included in this work. Though our reverse engineering efforts and formal model cannot directly apply to these services, we believe that the overall understanding and the vulnerabilities of the proximity tracking protocol recovered in this work can provide insights for researchers when conducting research on other proximity tracking protocols.

8 Related Work

BLE proximity tracking service. Before Apple and Samsung introduced the BLE proximity tracking service, there were already many Bluetooth trackers, like Tile, Nut, and so on. Heiland and Compton [18] discovered and disclosed 12 vulnerabilities in TrackR Bravo, iTrack Easy, and Nut. Their analysis focused on the Bluetooth interface and forging finder identities to access data. Weller et al. [49] presented a detailed analysis of the security and privacy aspects of various commercial Bluetooth trackers, including Nut, Tile, and so on, and proposed a novel privacy-preserving location-tracking system called PrivateFind. Alexander Heinrich et al. [28] reverse-engineered the Apple Find My protocols, analyzed the Security and Privacy of Apple’s location tracking System, and released an open-source framework for locating personal Bluetooth devices using Apple’s Find My Network. In this study, they uncovered two design and implementation vulnerabilities that could lead to unauthorized access to location histories and location correlation attacks. Tingfeng Yu et al. [56, 57] reverse-engineered the Samsung Find My Mobile protocols and analyzed the Security and Privacy of Samsung’s location tracking System. In this study, they uncovered serial vulnerabilities like a small pool of privacy IDs, DFU device reboot, unintended pairing with a SmartTag, Unintended Silent Pairing with an Owner Device, and so on.

Hardware and firmware security of AirTag. Thomas Roth et al. [39] liberated AirTag from being limited like extracting firmware from a physical device, customizing the sound set of AirTag, cloning AirTag, and degrading AirTag firmware.

Moreover, they analyzed the protocol used between iPhones and AirTags and unlocked undocumented commands.

Anti-tracking Technologies and Anti-anti-tracking Technologies. In 2021, Travis Mayberry et al. [32] provided some tricks like frequent key rotations to bypass the alert procedure, which Apple introduced to mitigate the abuse of Apple Find My network for malicious tracking. There are also some physical methods, like removing the speaker of AirTag to prevent it from making sound [1] and degrading AirTag firmware to delay the alert time [39]. Moreover, Narmeen Shafqat et al. [46] demonstrate a novel possibility of developing a stealthy cloned AirTag capable of tracking victims directly on the Find My app while circumventing safety alerts on the victim’s device and propose practical solutions to mitigate stalking risks from cloned AirTags and enhance the existing anti-stalking safeguards for AirTags. Alexander Heinrich et al. [27] designed and developed the AirGuard, an anti-tracking application mainly designed to protect Android users from malicious tracking that leverages Apple Find My Network.

Formal analysis of other protocols. Researchers have also performed formal symbolic verification of other protocols, such as Bluetooth, TLS, and Signal [33]. Jianliang Wu et al. [54, 55] adopt a modular design to model different Bluetooth protocols, formally verify Bluetooth security, and discover one new vulnerability and one security issue. Before Jianliang Wu et al., there were several works of [6, 14, 16, 45, 54] formal verification of Bluetooth protocols. Bhargavan et al. use ProVerif to verify TLS 1.0 [11] and 1.3 [10]. Kobeissi et al. [29] also discover new weaknesses in the Signal protocol via ProVerif. Girol et al. [23] conduct a comprehensive formal analysis of the Noise framework [47] using Tamarin.

9 Conclusion

Many BLE proximity tracking networks have worked for a long time. However, there is currently no systematic analysis scheme for the security and privacy of BLE proximity tracking services. We have presented a general workflow and a systematic analysis scheme for the BLE proximity tracking service. By reverse-engineering and analyzing Samsung Find My Mobile and Apple Find My, we uncovered a series of vulnerabilities, including unauthorized rebinding, device tracking, and privacy leakage.

Acknowledgments

This work was supported by National Natural Science Foundation of China under Grant(No. 62372268), Major Scientific and Technological Innovation Projects of Shandong Province, China(No. 2024CXGC010114), Shandong Provincial Natural Science Foundation, China (No. ZR2022LZH013, No. ZR2021LZH007).

References

- [1] How to Remove the Speaker Coil from the Airtag (Make Your Airtag Very Silent!). <https://www.youtube.com/watch?v=sgGNShP9H8A>, 2021.
- [2] Find my network accessory specification. <https://developer.apple.com/find-my/>, 2020.
- [3] Airtag. <https://www.apple.com/airtag/>, 2022.
- [4] Apple Find My. <https://www.apple.com/icloud/find-my/>, 2023.
- [5] How to reset your AirTag. <https://support.apple.com/en-us/102577>, 2023.
- [6] Kenichi Arai and Toshinobu Kaneko. Formal verification of improved numeric comparison protocol for secure simple pairing in bluetooth using proverif. In *Proceedings of the International Conference on Security and Management (SAM)*, page 1. The Steering Committee of The World Congress in Computer Science, Computer ..., 2014.
- [7] Myrto Arapinis, Tom Chothia, Eike Ritter, and Mark Ryan. Analysing unlinkability and anonymity using the applied pi calculus. In *Proceedings of the 23rd IEEE Computer Security Foundations Symposium, CSF 2010, Edinburgh, United Kingdom, July 17-19, 2010*, pages 107–121. IEEE Computer Society, 2010.
- [8] Manuel Barbosa, Gilles Barthe, Karthik Bhargavan, Bruno Blanchet, Cas Cremers, Kevin Liao, and Bryan Parno. Sok: Computer-aided cryptography. In *42nd IEEE Symposium on Security and Privacy, SP 2021, San Francisco, CA, USA, 24-27 May 2021*, pages 777–795. IEEE, 2021.
- [9] Daniel J. Bernstein. Curve25519: New diffie-hellman speed records. In *Public Key Cryptography - PKC 2006, 9th International Conference on Theory and Practice of Public-Key Cryptography, New York, NY, USA, April 24-26, 2006, Proceedings, volume 3958 of Lecture Notes in Computer Science*, pages 207–228. Springer, 2006.
- [10] Karthikeyan Bhargavan, Bruno Blanchet, and Nadim Kobeissi. Verified models and reference implementations for the TLS 1.3 standard candidate. In *2017 IEEE Symposium on Security and Privacy, SP 2017, San Jose, CA, USA, May 22-26, 2017*, pages 483–502. IEEE Computer Society, 2017.
- [11] Karthikeyan Bhargavan, Cédric Fournet, Ricardo Corin, and Eugen Zalinescu. Verified cryptographic implementations for TLS. *ACM Trans. Inf. Syst. Secur.*, 15(1):3:1–3:32, 2012.
- [12] Binary Ninja is an interactive decompiler, disassembler, debugger, and binary analysis platform built by reverse engineers, for reverse engineers. <https://binary.ninja/>, 2023.
- [13] Bluetooth specifications. <https://www.bluetooth.com/specifications/>, 2024.
- [14] Richard Chang and Vitaly Shmatikov. Formal analysis of authentication in bluetooth device pairing. *Fcs-arspa07*, 45, 2007.
- [15] Vincent Cheval, Charlie Jacomme, Steve Kremer, and Robert Künnemann. SAPIC+: protocol verifiers of the world, unite! In Kevin R. B. Butler and Kurt Thomas, editors, *31st USENIX Security Symposium, USENIX Security 2022, Boston, MA, USA, August 10-12, 2022*, pages 3935–3952. USENIX Association, 2022.
- [16] Cas Cremers and Dennis Jackson. Prime, order please! revisiting small subgroup and invalid curve attacks on protocols using diffie-hellman. In *2019 IEEE 32nd Computer Security Foundations Symposium (CSF)*, pages 78–7815. IEEE, 2019.
- [17] DeepSec prover. <https://deepsec-prover.github.io/manual/>, 2024.
- [18] Multiple Bluetooth Low Energy (BLE) Tracker Vulnerabilities. <https://blog.rapid7.com/2016/10/25/multiple-bluetooth-low-energy-ble-tracker-vulnerabilities/>, 2016.
- [19] Apple AirTag Reverse Engineering. <https://adamcattley.com/AirTag.html#supported-devices>, 2022.
- [20] Danny Dolev and Andrew Chi-Chih Yao. On the security of public key protocols. *IEEE Trans. Inf. Theory*, 29(2):198–207, 1983.
- [21] Frida - Dynamic instrumentation toolkit for developers, reverse-engineers, and security researchers. <https://frida.re/>, 2023.
- [22] Chinmay Garg, Aravind Machiry, Andrea Continella, Christopher Kruegel, and Giovanni Vigna. Toward a secure crowdsourced location tracking system. In *Proceedings of the 14th ACM Conference on Security and Privacy in Wireless and Mobile Networks*, pages 311–322, 2021.
- [23] Guillaume Girol, Lucca Hirschi, Ralf Sasse, Dennis Jackson, Cas Cremers, and David A. Basin. A spectral analysis of noise: A comprehensive, automated, formal analysis of diffie-hellman protocols. In *29th USENIX Security Symposium, USENIX Security 2020, August 12-14, 2020*, pages 1857–1874. USENIX Association, 2020.

- [24] jadx - Dex to Java decompiler. <https://github.com/skylot/jadx>, 2023.
- [25] Github BLETracker. <https://github.com/xiaofeng-L/BLETracker.git>, 2025.
- [26] Android Studio. <https://developer.android.com/studio?hl=zh-cn>, 2024.
- [27] Alexander Heinrich, Niklas Bittner, and Matthias Hollick. Airguard - protecting android users from stalking attacks by apple find my devices. In WiSec '22: 15th ACM Conference on Security and Privacy in Wireless and Mobile Networks, San Antonio, TX, USA, May 16 - 19, 2022, pages 26–38. ACM, 2022.
- [28] Alexander Heinrich, Milan Stute, Tim Kornhuber, and Matthias Hollick. Who can find my devices? security and privacy of apple’s crowd-sourced bluetooth location tracking system. Proc. Priv. Enhancing Technol., 2021(3):227–245, 2021.
- [29] Nadim Kobeissi, Karthikeyan Bhargavan, and Bruno Blanchet. Automated verification for secure messaging protocols and their implementations: A symbolic and computational approach. In 2017 IEEE European Symposium on Security and Privacy, EuroS&P 2017, Paris, France, April 26-28, 2017, pages 435–450. IEEE, 2017.
- [30] John Koetsier. How to track people with apple airtags. <https://www.forbes.com/sites/johnkoetsier/2021/04/22/how-to-track-people-with-apple-airtags/?sh=2845a94169df>, April 2021.
- [31] nRF52 Debug Resurrection (APPROTECT Bypass) Part 1. <https://limitedresults.com/2020/06/nrf52-debug-resurrection-appprotect-bypass/>, 2020.
- [32] Travis Mayberry, Ellis Fenske, Dane Brown, Jeremy Martin, Christine Fossaceca, Erik C. Rye, Sam Teplov, and Lucas Foppe. Who tracks the trackers?: Circumventing apple’s anti-tracking alerts in the find my network. In WPES '21: Proceedings of the 20th Workshop on Workshop on Privacy in the Electronic Society, Virtual Event, Korea, 15 November 2021, pages 181–186. ACM, 2021.
- [33] Signal on the outside, Signal on the inside. <https://signal.org/blog/signal-inside-and-out/>, 2016.
- [34] Official Documentation. <https://infocenter.nordicsemi.com/index.jsp>, 2022.
- [35] Andreas Pfitzmann and Marit Hansen. A terminology for talking about privacy by data minimization: Anonymity, unlinkability, undetectability, unobservability, pseudonymity, and identity management, 2010.
- [36] Burp Suite - The class-leading vulnerability scanning, penetration testing, and web app security platform. <https://portswigger.net/burp>, 2023.
- [37] ProVerif 2.04: Automatic Cryptographic Protocol Verifier, User Manual and Tutorial. <https://bblanche.gitlabpages.inria.fr/proverif/manual.pdf>, 2021.
- [38] USB dongle. <https://www.szrfstar.com/product/221-cn.html>, 2024.
- [39] Thomas Roth, Fabian Freyer, Matthias Hollick, and Jiska Classen. Airtag of the clones: Shenanigans with liberated item finders. In 43rd IEEE Security and Privacy, SP Workshops 2022, San Francisco, CA, USA, May 22-26, 2022, pages 301–311. IEEE, 2022.
- [40] [Update] Introducing the New Galaxy SmartTag+: The Smart Way to Find Lost Items. <https://news.samsung.com/us/introducing-the-new-galaxy-smart-tag-plus/>, 2023.
- [41] Samsung SmartThings Find Hits New Milestone With 200 Million Nodes Helping Find Lost Devices. <https://news.samsung.com/global/samsung-smarththings-find-hits-new-milestone-with-200-million-nodes-helping-find-lost-devices>, 2022.
- [42] How to reset the Samsung SmartTag? <https://www.samsung.com/ae/support/mobile-devices/how-to-reset-the-samsung-smarttag/>, 2022.
- [43] SmartThings Find. <https://www.samsung.com/global/galaxy/apps/smarththings-find/>, 2023.
- [44] Use the Samsung Galaxy SmartTag and SmartTag+. <https://www.samsung.com/us/support/answer/ANS00088244/>, 2023.
- [45] Mohit Sethi, Aleks Peltonen, and Tuomas Aura. Misbinding attacks on secure device pairing and bootstrapping. In Proceedings of the 2019 ACM Asia Conference on Computer and Communications Security, pages 453–464, 2019.
- [46] Narmeen Shafqat, Nicole Gerzon, Maggie Van Nortwick, Victor Sun, Alan Mislove, and Aanjan Ranganathan. Track you: A deep dive into safety alerts for apple airtags. Proc. Priv. Enhancing Technol., 2023(4):132–148, 2023.
- [47] The Noise Protocol Framework. <https://noiseprotocol.org/noise.html>, 2018.
- [48] Tamarin Prover. <https://tamarin-prover.com/>, 2023.

- [49] Mira Weller, Jiska Classen, Fabian Ullrich, Denis Waßmann, and Erik Tews. Lost and found: stopping bluetooth finders from leaking private information. In *WiSec '20: 13th ACM Conference on Security and Privacy in Wireless and Mobile Networks*, Linz, Austria, July 8-10, 2020, pages 184–194. ACM, 2020.
- [50] Haohuang Wen, Zhiqiang Lin, and Yinqian Zhang. Firmxray: Detecting bluetooth link layer vulnerabilities from bare-metal firmware. In Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna, editors, *CCS '20: 2020 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, USA, November 9-13, 2020*, pages 167–180. ACM, 2020.
- [51] Extended Euclidean algorithm. https://en.wikipedia.org/wiki/Extended_Euclidean_algorithm, 2022.
- [52] Fermat's little theorem. https://en.wikipedia.org/wiki/Fermat%27s_little_theorem, 2022.
- [53] Wireshark - The world's most popular network protocol analyzer. <https://www.wireshark.org/>, 2023.
- [54] Jianliang Wu, Yuhong Nan, Vireshwar Kumar, Dave Jing Tian, Antonio Bianchi, Mathias Payer, and Dongyan Xu. BLESa: Spoofing Attacks against Reconnections in Bluetooth Low Energy. In *14th USENIX Workshop on Offensive Technologies (WOOT 20)*, 2020.
- [55] Jianliang Wu, Ruoyu Wu, Dongyan Xu, Dave Jing Tian, and Antonio Bianchi. Formal model-driven discovery of bluetooth protocol design vulnerabilities. In *43rd IEEE Symposium on Security and Privacy, SP 2022, San Francisco, CA, USA, May 22-26, 2022*, pages 2285–2303. IEEE, 2022.
- [56] Tingfeng Yu, James Henderson, Alwen Tiu, and Thomas Haines. Security and privacy analysis of samsung's crowd-sourced bluetooth location tracking system.
- [57] Tingfeng Yu, James Henderson, Alwen Tiu, and Thomas Haines. Privacy analysis of samsung's crowd-sourced bluetooth location tracking system. *CoRR*, abs/2210.14702, 2022.

Appendix A Proof of V2

We call $E(X)$ the mathematical expectation of the number of different IDs in X IDs set. We assume that the randomized ID is completely randomly generated. Then,

$$E(X) = 1000 * (1 - 0.999^X)$$

We calculate the average time for attackers to collect 80 percent of IDs is close to three weeks, since the number of

different IDs collected for two weeks can reach $E(1,344) = 739.37$ and the number for three weeks can reach $E(2,016) = 866.95$. To support our proof, we perform a collection experiment, which is shown in our github [25].

If the ID entropy is changed to 65,535, attackers need 156.5 weeks to collect 80% IDs, as $E(105,200) = 52,428$.

$$E1(X) = 65535 * (1 - (1 - 1/65535)^X)$$

Appendix B Proof of V6

Samsung Find My Mobile employs the elliptic curve Curve25519 [9], which was designed to achieve high speeds at computation without compromising the security strength. It is defined over the prime p field of order $2^{255} - 19$ with a generator point $G = 9$. Since p is prime, the set $1, \dots, p-1$ is a multiplicative group modulo p . As a group of prime order must be a cyclic group, according to Fermat's little theorem [52], there must exist a multiplicative inverse a^{-1} for P . Finally, the attacker can use the Extended Euclidean algorithm [51] to compute a^{-1} in a short time.

We use (A, a) to present the public-private key pair corresponding hash value *hashed_sn1*, (B, b) to present the public-private key pair corresponding hash value *hashed_sn2*, (C, c) to present the orient cloud public-private key pair, and (D, d) to present the fake cloud public-private key pair. Then, we prove the equivalence of the master secrets calculated by the Samsung server and the SmartTag as follows.

$$\begin{aligned} D &= a^{-1} \cdot (b \cdot c) \cdot G, & DHKey_{cloud} &= b \cdot C = (b \cdot c) \cdot G \\ DHKey_{tag} &= a \cdot D = (a \cdot a^{-1} \cdot c \cdot b) \cdot G = (b \cdot c) \cdot G \\ masterSecret_{cloud} &= SHA256(DHKey_{cloud} || rand) \\ masterSecret_{tag} &= SHA256(DHKey_{tag} || rand) \end{aligned}$$

Appendix C Apple Find My in Detail

Initialization. For ease of explanation, as illustrated in Figure 7, we refer to the iPhone as P , the Apple tracker (i.e., AirTag) as T , and the Apple server as S . In step 1, the iPhone builds a BLE connection with the Apple tracker via the secure pairing mode of BLE Just Works. Both the iPhone and the tracker will generate a shared session key used to encrypt subsequent data transmission.

In step 2, initially, P generates a nonce *SessionNonce* and an encrypted blob $E1$ and sends *SessionNonce* and $E1$ to T . After receiving these, T generates a P-224 scalar s and a random value r and computes the hash value $C1$ as described in Equation(6). Then, T generates a random value *SeesK1* and calculates the encrypted blob $E2$ using the Apple Server encryption key Q_E as described in Equation(7), which includes the *SessionNonce*, its origin software authentication token *token*, the *UUID* (associated with the token), its serial

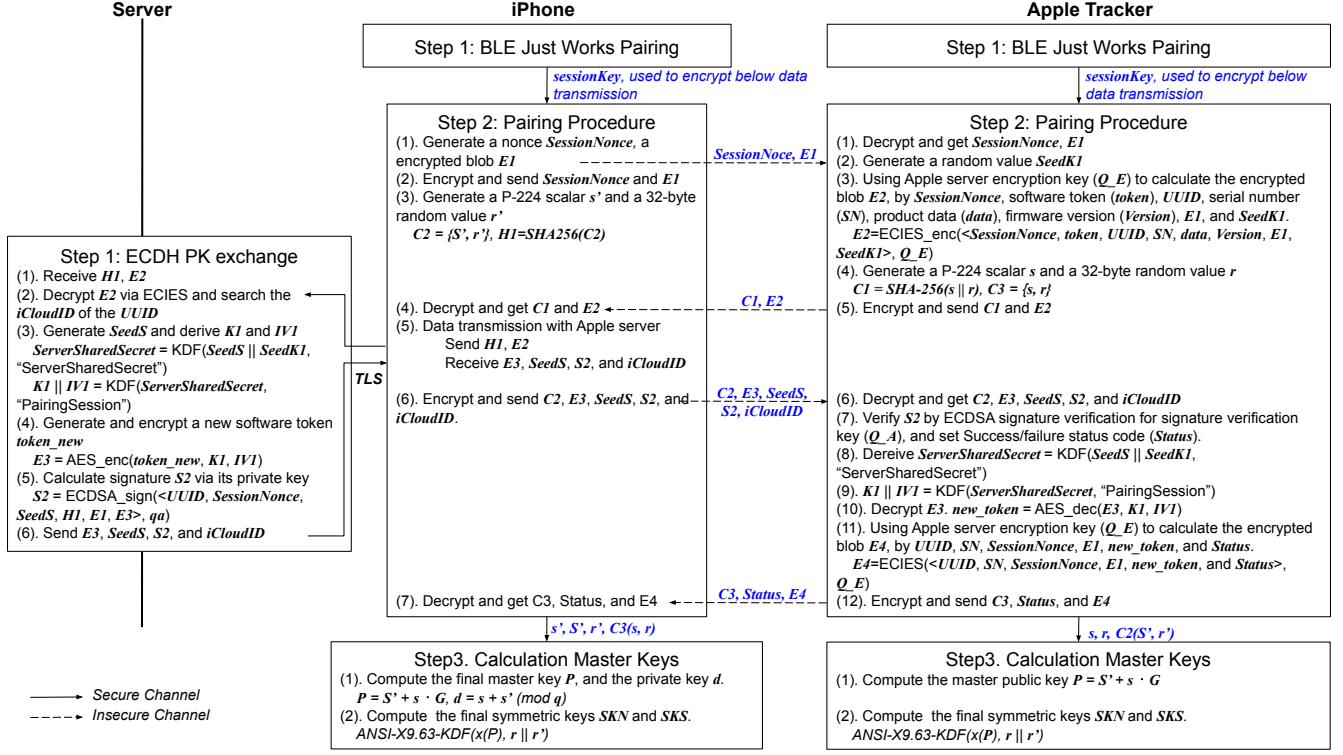


Figure 7: The initialization phase of Apple Find My.

number SN , its product data, its firmware version, $E1$, and $SeedK1$. Subsequently, T sends $C1$ and $E2$ to P .

$$C3 = (s, r), C1 = SHA256(C3) \quad (6)$$

$$E2 = ECIES_enc(< SessionNonce, token, UUID, SN, data, version, E1, SeedK1 >, Q_E) \quad (7)$$

$$S' = s' \cdot G, \quad C2 = (S', r') \quad (8)$$

$$H1 = SHA256(C2) \quad (9)$$

After receiving $C1$ and $E2$, P generates its private key s' and a random value r' . Then, P calculates its corresponding public key S' as described in Equation(8) and the hash value $H1$ as described in Equation(9), and sends $H1$ and $E2$ to S .

After receiving $H1$ and $E2$, S decrypts $E2$. If S verifies that the software authentication token and $UUID$ is valid, S will generate a random value $SeedS$ and derive the server shared secret SSS as described in Equation(11) via ANSI-X9.63-KDF, $K1$ and $IV1$ as described in Equation(12). Then, S will provide a new software authentication token $E3$ which is encrypted by $K1$ and $IV1$ as described in Equation(C). Subsequently, S uses its private key to generate the signature $S2$ as described in Equation(13), and sends the iPhone's $iCloud$ identifier $iCloudID$, $SeedS$, $S2$, and $E3$. After P receiving these, P will send $C2$, $E3$, $SeedS$, $S2$, and $iCloudID$ to T .

After receiving the information above, T first verifies $S2$ by the signature verification key Q_A and sets the verification result into $Status$. If $Status$ is *Success*, T calculates

$ServerSharedSecret$, $K1$, and $IV1$ to decrypt $E3$ as described in Equation(14). After that, similar to computing $E2$, T computes the encrypted blob $E4$ as described in Equation(15). Subsequently, T sends $C3$, $Status$, and $E4$ to P .

$$SSS = KDF(SeedS || SeedK1, 'ServerSharedSecret') \quad (10)$$

$$K1 || IV1 = KDF(SSS, 'PairingSession') \quad (11)$$

$$E3 = AES_GCM_enc(token_new, K1, IV1) \quad (12)$$

$$S2 = ECDSA_sign(< UUID, SessionNonce, SeedS, H1, E1, E3 >, qa) \quad (13)$$

$$token_new = AES_GCM_dec(E3, K1, IV1) \quad (14)$$

$$E4 = ECIES_enc(< UUID, SN, SessionNonce, E1, new_token, and Status >, Q_E) \quad (15)$$

Finally, in step (3), both P and T compute the master public key P as described in Equation(16) and the master symmetric key ($SKN || SKS$) as described in Equation(17). $X(P)$ is used to get the X point's value of a given P-224 elliptic curve point P . However, only P can compute the master private key d as described in Equation(18).

$$P = S' + s \cdot G \quad (16)$$

$$(SKN || SKS) = KDF(X(P), r || r') \quad (17)$$

$$d = s + s' \quad (18)$$